

QUANDRIX

Marketplace de cartas Magic: The Gathering

Reporte de Pruebas Unitarias

Arquitectura de Microservicios — Backend Spring Boot

Asignatura: Desarrollo Fullstack I

Programa: Analista Programador

Autora: Ignacia Padilla

Fecha: 29 de junio de 2026

1. Resumen ejecutivo

Este documento resume el proceso de pruebas unitarias aplicado a los 10 microservicios de negocio del proyecto Quandrix, un marketplace de cartas Magic: The Gathering construido sobre una arquitectura de microservicios con Spring Boot 4.0.7, Spring Cloud 2025.1.1, Eureka y API Gateway.

Siguiendo la metodología entregada por el profesor de la asignatura (análisis por clase, estructura ARRANGE/ACT/ASSERT/VERIFY, ejecución vía Maven y revisión de reportes Surefire), se generaron pruebas unitarias para las capas de Controller y Service de cada microservicio, alcanzando un total de:

270 pruebas unitarias

distribuidas en 10 microservicios, con 0 fallas y 0 errores en la ejecución final de cada uno.

1.1 Cobertura por microservicio

Microservicio	Clases cubiertas	Tests (Controller / Service)	Total
ms-auth	AuthController / AuthService / JwtService	6 / 8 / 5	19
ms-catalog	CatalogController / CatalogService / ScryfallClient	8 / 10 / 7	25
ms-listings	ListingController / ListingService	18 / 24	42
ms-notifications	NotificationController / NotificationService	13 / 16	29
ms-reports	ReportController / ReportService	9 / 12	21
ms-orders	OrderController / OrderService	13 / 18	31
ms-payments	PaymentController / PaymentService	7 / 7	14
ms-reviews	ReviewController / ReviewService	13 / 15	28
ms-transactions	TransactionController / TransactionService	13 / 10	23
ms-users	UserProfile + StoreProfile (Controller/Service)	20 / 18	38
TOTAL	10 microservicios de negocio		270

Nota: la columna “Tests (Controller / Service)” indica la cantidad de pruebas por clase principal, en el orden en que aparecen en la columna “Clases cubiertas”. En microservicios con clientes externos con lógica propia (ScryfallClient en ms-catalog) se incluye una tercera cifra.

2. Metodología aplicada

Para cada microservicio se siguió un proceso de 5 pasos:

- Paso 1 — Análisis: revisión de la clase real (controller o service), identificación de dependencias a simular y mapeo de las ramas de ejecución (casos de éxito y de falla).
- Paso 2 — Preparación de carpeta: creación de la estructura de test espejo a la de producción (src/test/java/...).
- Paso 3 — Codificación: escritura del test siguiendo la estructura ARRANGE / ACT / ASSERT / VERIFY, con un comentario de “caso hipotético de falla” en cada prueba, redactado desde la perspectiva de control de calidad (QA).
- Paso 4 — Ejecución: corrida del test vía Maven (mvn -pl <módulo> -Dtest=<Clase> test).
- Paso 5 — Reporte: verificación del resultado en los reportes generados por Surefire.

Herramientas utilizadas:

- JUnit 5 + Mockito — pruebas de la capa Service, simulando repositorios y clientes Feign con `@Mock` / `@InjectMocks`.
- MockMvc + `@WebMvcTest` + `@MockitoBean` (Spring Boot 4) — pruebas de la capa Controller, simulando el Service y verificando códigos HTTP, cuerpos de respuesta y delegación correcta.
- MockWebServer (OkHttp) — único caso de un cliente externo con lógica propia (ScryfallClient en ms-catalog, basado en WebClient), donde se levantó un servidor HTTP de prueba real en lugar de simular la cadena de llamados de WebClient.
- AssertJ — aserciones legibles (assertThat...) en las pruebas de Service.

2.1 Compatibilidad con Spring Boot 4

Al ser Quandrix un proyecto sobre Spring Boot 4.0.7 (no 3.x), se identificaron y resolvieron de forma temprana los siguientes cambios de API respecto a versiones anteriores, evitando que se repitiera la misma depuración en cada microservicio:

- `@MockBean` fue reemplazado por `@MockitoBean` (paquete `org.springframework.test.context.bean.override.mockito`).
- `@WebMvcTest` cambió de paquete: ahora vive en `org.springframework.boot.webmvc.test.autoconfigure`.
- Se requiere agregar explícitamente la dependencia `spring-boot-webmvc-test` (scope `test`) en cada módulo que prueba un controller, ya que `spring-boot-starter-test` por sí solo no la incluye.

2.2 Criterios de diseño consistentes

A lo largo de los 10 microservicios se aplicaron, de forma deliberada, los siguientes criterios:

- Lista vacía no es error: en endpoints de listado (getBySeller, getByBuyer, getAllActive, etc.), cuando el Swagger documentaba un 404 que el código real nunca lanza, se decidió documentar el comportamiento real (200 OK + lista vacía) como “aclaración de contrato”, en lugar de forzar artificialmente un 404 inexistente.
- Separación de responsabilidades en clientes Feign: las interfaces `@FeignClient` sin lógica propia (CatalogClient, UserClient, PaymentClient, TransactionClient, NotificationClient, AuthClient) se simulan directamente en las pruebas de Service, sin necesidad de un archivo de prueba propio. Solo se generó un test dedicado para

ScryfallClient, por tratarse de una clase con lógica de traducción de errores HTTP propia, usando MockWebServer.

- Manejo de no determinismo: en ms-payments, la simulación de aprobación/rechazo de pagos incluye un componente aleatorio (10% de probabilidad). Se diseñaron las pruebas para afirmar únicamente lo que es determinístico en cada escenario, utilizando el campo forceFailure expuesto por el propio código para aislar con certeza el camino de rechazo.
- Documentación de discrepancias Swagger vs. código real: cuando la documentación de la API (Swagger) describía un comportamiento que el código no implementaba (por ejemplo, códigos de error documentados que nunca se disparan), se dejó constancia en el comentario del test correspondiente, sin alterar el comportamiento real del sistema sin autorización del equipo de desarrollo.

3. Hallazgos relevantes durante las pruebas

El proceso de testing no solo verificó el comportamiento esperado del sistema, sino que permitió identificar los siguientes hallazgos, agrupados por categoría. Ninguno de estos hallazgos fue corregido unilateralmente: cada uno quedó documentado en el código de prueba correspondiente para que el equipo de desarrollo decida la acción a seguir.

3.1 Hallazgos de seguridad

Microservicio	Hallazgo
ms-transactions	El endpoint GET /transactions/admin, documentado como "Solo ADMIN", no tiene ninguna restricción de rol real: SecurityConfig usa .anyRequest().permitAll(). Cualquier usuario sin autenticar puede consultar todas las transacciones del marketplace (montos, comprador, vendedor). Se recomienda confirmar si el API Gateway efectivamente bloquea esta ruta antes de considerar el hallazgo resuelto.
ms-listings	Los métodos update() y withdraw() de ListingService validan correctamente que el sellerId del request coincida con el dueño real del listing (protección contra IDOR confirmada por test), pero se documentó explícitamente el riesgo de que esta validación se elimine por error en una futura modificación.
ms-orders	El método cancel() de OrderService incluye la misma protección IDOR (un comprador no puede cancelar una orden ajena), validada con test dedicado.

3.2 Hallazgos de diseño y manejo de errores

Microservicio	Hallazgo
ms-reviews	ReviewService.create() lanza IllegalArgumentException (no una excepción de negocio propia) cuando el rating está fuera de rango. Como GlobalExceptionHandler no tiene un manejador dedicado para ese tipo, si el código llegara a alcanzarse, el resultado sería un HTTP 500 en lugar de un 400. En la práctica, Bean Validation (@Min/@Max en el DTO) intercepta el request antes de llegar al service, por lo que el defecto es real pero no explotable vía la API HTTP normal actual.

Microservicio	Hallazgo
ms-catalog	El método <code>getAllSets()</code> usa <code>cardRepository.findAll()</code> en lugar de <code>cardSetRepository.findAll()</code> , retornando cartas individuales en lugar de sets/ediciones, a pesar de lo que sugiere su nombre y su documentación Swagger. Tras revisión conjunta, se decidió mantener el comportamiento actual (no se trata como bug a corregir).
ms-notifications	El mensaje de <code>InvalidNotificationTypeException</code> lista valores en español (NUEVA_ORDEN, PAGO_CONFIRMADO, etc.) que no corresponden a ningún valor real del enum <code>NotificationType</code> (en inglés). Un consumidor de la API que corrija su request según ese mensaje seguiría recibiendo el mismo error.
ms-orders	El método <code>create()</code> distingue explícitamente entre pasos críticos (validación de listing, procesamiento de pago) y pasos tolerantes a fallos (marcar listing como vendido, registrar transacción, notificar). Se identificó que, si un paso tolerante fallara sin ser capturado, el rollback de <code>@Transactional</code> revertiría solo la base de datos local, no las llamadas Feign ya ejecutadas hacia otros microservicios — un riesgo arquitectónico documentado, no un defecto del código de prueba.

3.3 Discrepancias de documentación (Swagger vs. comportamiento real)

Microservicio	Hallazgo
ms-payments	<code>PaymentProcessingException</code> retorna HTTP 422 Unprocessable Entity; el Swagger del controller documenta 400.
ms-transactions / ms-reports	Los endpoints de rango de fechas documentan un posible 400 (“fechas inválidas”) que el código nunca implementa; cualquier from/to es aceptado sin validar <code>from ≤ to</code> .
Múltiples microservicios	Varios endpoints de listado (<code>getBySeller</code> , <code>getByBuyer</code> , <code>getAllActive</code> , <code>getByCard</code> , etc.) documentan un 404 “sin resultados” que el código nunca lanza; el comportamiento real es 200 OK con lista vacía.

4. Conclusiones

El conjunto de 270 pruebas unitarias generadas cubre, para los 10 microservicios de negocio de Quandrix, la totalidad de los métodos públicos de sus capas de Controller y Service, incluyendo:

- Casos de éxito (camino feliz) para cada operación.
- Casos de error de validación, tanto de Bean Validation (capa Controller) como de reglas de negocio (capa Service).
- Casos de borde: listas vacías, valores nulos, duplicados, recursos inexistentes.
- Casos de tolerancia a fallos, en los flujos de orquestación más complejos (ms-orders).
- Tests de integración acotada con servidores HTTP simulados (MockWebServer), para el único cliente externo con lógica propia.

Quedan fuera del alcance de este trabajo, por tratarse de pruebas de un tipo distinto al unit testing por capas solicitado:

- Pruebas de integración real entre microservicios (Testcontainers, contract testing).
- Pruebas de carga o concurrencia.
- Pruebas sobre clases de configuración pura de Spring (SecurityConfig, FeignConfig, etc.), sin lógica condicional propia.
- Pruebas sobre interfaces @FeignClient sin lógica propia, simuladas directamente en las pruebas de Service que las consumen.

Los hallazgos documentados en la sección 3 no fueron corregidos como parte de este trabajo de testing, siguiendo el principio de que la responsabilidad de las pruebas es verificar y documentar el comportamiento del sistema, no modificarlo unilateralmente. Se entregan como insumo para que el equipo de desarrollo decida las correcciones pertinentes.